

EXTREMAL GRAPH THEORY

Erdős Problem 915

Maximising Edge Density Subject to Connectivity Constraints

Louis Vandenbruwaene

Supervisor: Prof. Stijn Cambie

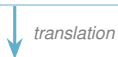
Promotor: Prof. Jan Goedgebeur

KU Leuven, Master of Mathematics

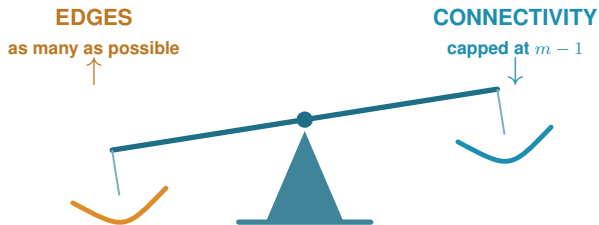
KU LEUVEN

The title, in plain words

Maximising edge density in graphs
under connectivity constraints

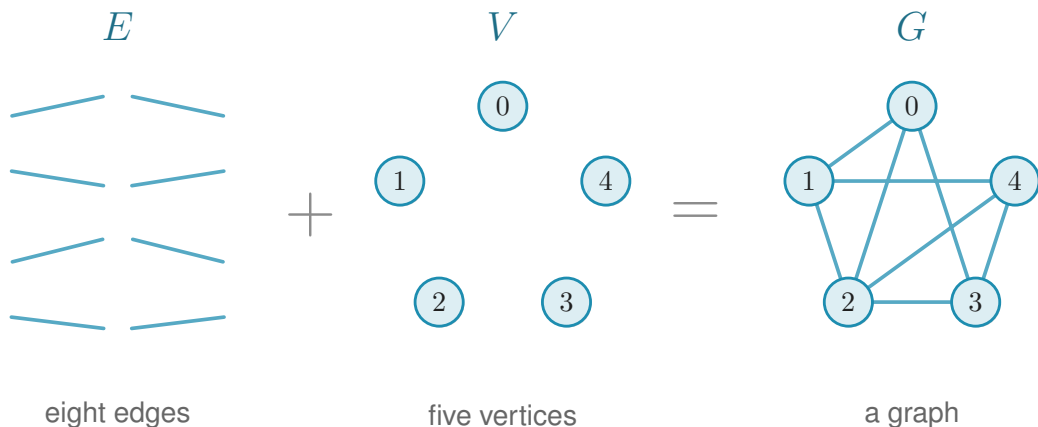


networks with **many links** and **low connectivity**



Push the edge count up while holding every pair of vertices weakly joined. The whole thesis is that one trade-off, asked over and over in different models.

A graph is edges and vertices



An edge joins a pair of vertices, and $|E(G)|$ is the number being maximised. The picture is $K_5 - \{04, 13\}$, the running example: five vertices, eight edges.

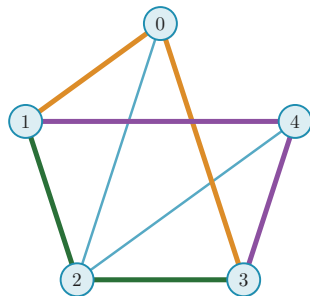
Erdős Problem 915

as many edges as possible, subject to the cap

$$\ell_m(n) = \max\{|E(G)| : \lambda_G^{\max} \leq m-1\}$$

$$\lambda_G^{\max} = \max_{u \neq v} \lambda_G(u, v) \text{ over every pair of vertices}$$

$\lambda_G(u, v)$ = the largest number of u - v routes that share **no edge**: one colour each



$$\lambda(1, 3) = 3$$

three routes from 1 to 3,
three colours, no edge reused

Menger's theorem gives the same number a second face: $\lambda_G(u, v)$ is also the fewest edges whose removal cuts every u - v route. That is the face the program will measure.

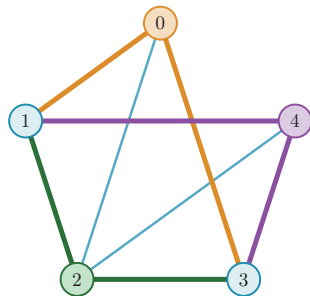
The same question, vertex by vertex

the same cap, read on vertices instead of edges

$$k_m(n) = \max \{ |E(G)| : \kappa_G^{\max} \leq m - 1 \}$$

$$\kappa_G^{\max} = \max_{u \neq v} \kappa_G(u, v) \text{ over every pair of vertices}$$

$\kappa_G(u, v)$ = the largest number of u - v routes that share **no interior vertex**

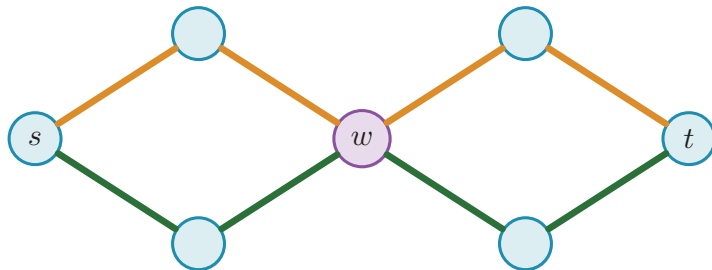


$$\kappa(1, 3) = 3$$

each route keeps its own interior
vertex: 0, 2 and 4

Whitney's inequality $\kappa_G(u, v) \leq \lambda_G(u, v)$: avoiding shared vertices is at least as demanding as avoiding shared edges. Here the two agree, but they need not.

Where the two part company

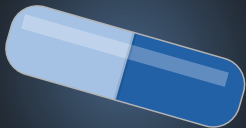


$$\lambda(s, t) = 2$$

$$\kappa(s, t) = 1$$

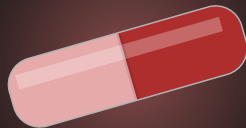
Two routes that share no edge are still forced through the single vertex w . Every toggle in this thesis is chosen because it moves the answer, and this one does.

THREE CHOICES TO MAKE



the graph you already know

simple, non-hyper, undirected



seven more models

toggle multiplicity, arity, direction

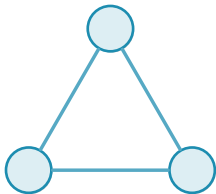
$2 \text{ multiplicity} \times 2 \text{ arity} \times 2 \text{ direction} = 8 \text{ models}$, and each is asked twice, for λ and for κ : **16 variants**.

Choice 1 of 3: multiplicity



simple

at most one edge per pair



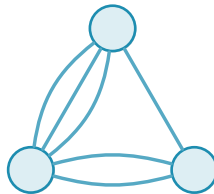
$$\mu(u, v) \in \{0, 1\}$$

VS



multigraph

parallel copies allowed

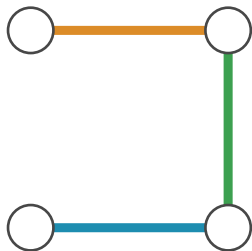


$$\mu(u, v) \in \{0, \dots, m-1\}$$

A parallel copy is a route of its own, so the cap is spent on multiplicity before it is spent on new neighbours. The extremal function is written $L_m(n)$ rather than $\ell_m(n)$.

Choice 2 of 3: arity

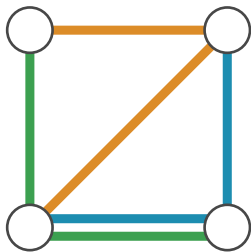
 **graph**
every line serves two stops



$$r = 2$$

VS

 **hypergraph**
a line serves three or more



$r \geq 2$, hyperedges span r vertices

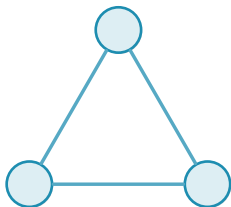
Routes become Berge paths, riding a hyperedge from any member to any other. The encoding changes with the model: a hypergraph is stored as a list of hyperedges, not as a matrix.

Choice 3 of 3: direction



undirected

an edge is an unordered pair



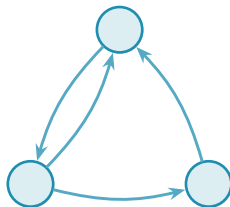
value linear in n

VS



directed

an arc is an ordered pair



value quadratic in n

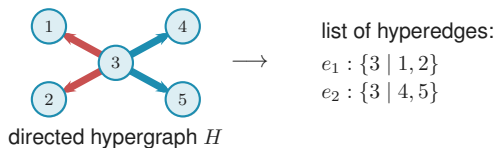
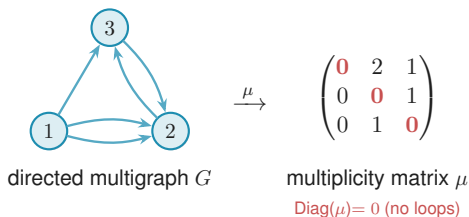
One-way traffic is the toggle that changes the order of the answer: a route from u to v no longer doubles as a route from v to u , so a graph can hold on the order of $n^2/4$ arcs under the same cap. These are the open cases.

PART II

The program

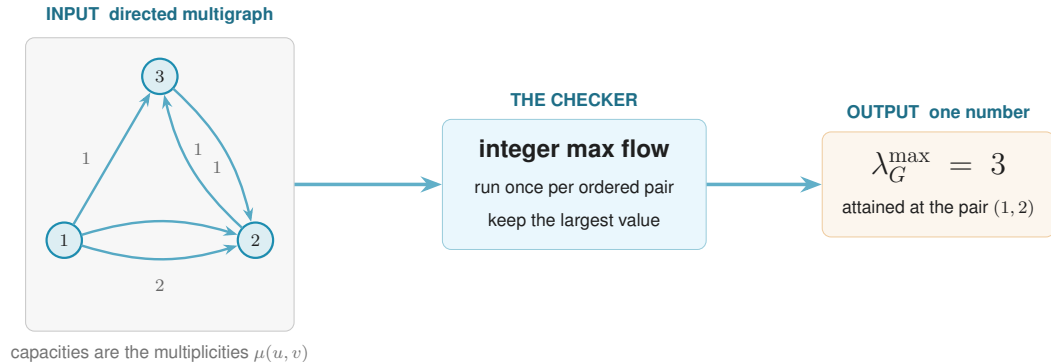
Sixteen variants, one flow computation.

Modelling: two encodings



A matrix would be too large and too sparse for hyperedges, so the model splits in two: every non-hypergraph is one multiplicity matrix, symmetric when undirected and binary when simple, and every hypergraph is a list of vertex sets, with the tail marked when directed.

Measuring: one max-flow computation



$$\lambda_G(u, v) \stackrel{\text{Menger}}{=} \text{smallest } u-v \text{ cut} \stackrel{\text{Ford-Fulkerson}}{=} \text{largest } u-v \text{ flow}$$

So the connectivity question is a flow question, and the whole program has one numerical primitive. Every other variant is transformed until it becomes this input.

Transformations: everything becomes this one input

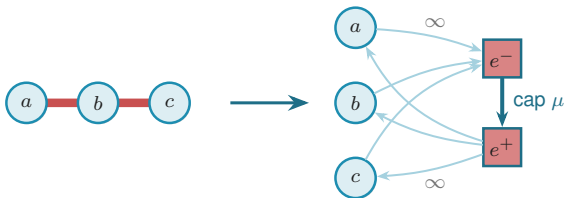
undirected $\rightarrow$ directed

one edge becomes two
arcs



hypergraph $\rightarrow$ graph

a hyperedge becomes a
gate



$\kappa \rightarrow \lambda$

a vertex becomes a gate
too

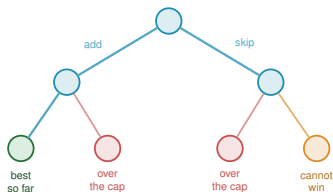


A gate is the only bottleneck on the routes through it, so its capacity is exactly what a route must spend. The three are applied in this order, and the gates themselves are never split.

Efficiency: three ways to make the sweep finish

Pruning

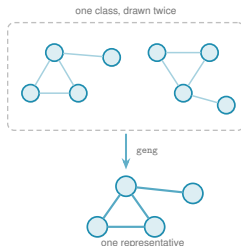
cut a branch, not a graph



Adding an edge never lowers connectivity, so a partial graph already over the cap condemns every completion.

Generating search

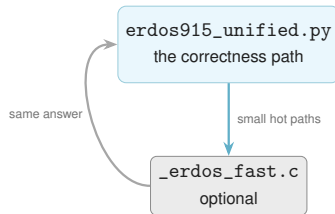
do not check the same graph twice



nauty's geng lists one support per class; the program then decorates its edges with arc multiplicities.

A compiled inner loop

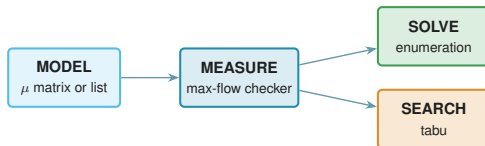
the same flow, faster



Capacities too large for safe `int32` arithmetic fall back to the Python integer path, so a speed-up cannot change an answer.

None of the three changes what is proved. Only infeasible branches and branches that cannot beat the incumbent are discarded, so a sweep that finishes returns exactly what the blind walk would have returned.

Solving versus searching



Solving

walk the whole space

- every graph on n vertices, measured
- **finishes** $\Rightarrow$ the exact value at that n , upper bound included
- **times out** $\Rightarrow$ only the best graph seen, a lower bound

Searching

steer through the space

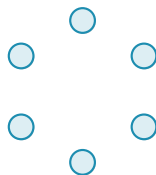
VS

- tabu search on the energy Φ , from the empty graph
- reaches sizes the sweep cannot
- **always** exhibits a graph, so **always** only a lower bound

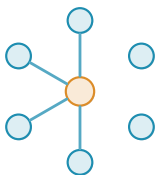
A value is settled only where a proof and a construction meet.

The search: one number per graph

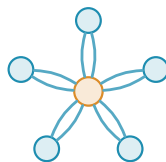
$$\Phi(G) = \underbrace{-|E(G)|}_{\substack{\text{reward} \\ \text{more edges, lower energy}}} + \underbrace{\Lambda \cdot \max(0, c^{\max}(G) - (m - 1))}_{\substack{\text{penalty} \\ \Lambda \text{ large, so breaking the cap never pays}}}$$



$$\Phi = 0$$



$$\Phi = -4$$



$$\Phi = -10$$

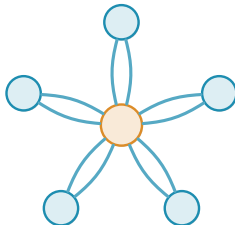
Tabu search, started from the empty graph on $n = 6$ at $m = 3$, walks downhill in Φ and forbids recently touched edges so it cannot walk straight back. Its endpoint here, the multiplicity-2 star with ten edges, is the known extremiser $L_3(6) = 10$.

What the search finds on its own



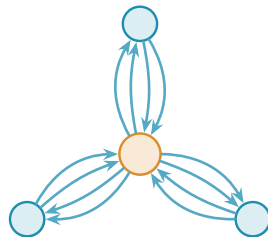
spanning tree

$$n = 6, \ell_2(n) = 5$$



multigraph star

$$n = 6, m = 3, L_3(n) = 10$$

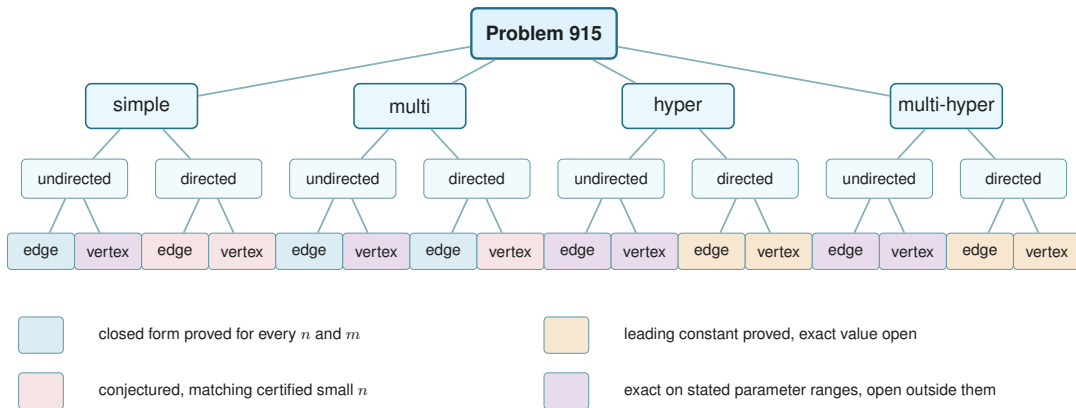


double star

$$n = 4, m = 3, L_3^{\text{dir}}(n) = 12$$

Told only to pack in edges, the search returns the named constructions. That is the check on the machine. It still only ever exhibits a graph, so it proposes a lower bound and never an upper one.

Status of the sixteen variants



Every leaf is one extremal function and its colour is that leaf's status. The open and conjectural leaves sit in the directed half of the tree.

What is new here

1 One-way multigraphs, solved for every n and every m

$L_m^{\text{dir}}(n) = (m-1) \max\left(2(n-1), \left\lfloor \frac{n^2}{4} \right\rfloor\right)$, by deleting a sparse reachability-preserving subgraph once per unit of $m-1$ rather than a vertex at a time. The quadratic extremisers are classified for $n \geq \max(8, 2m)$.

2 One-way simple graphs: an unconditional quadratic bound

$|A(D)| \leq \left\lfloor \frac{n^2}{4} \right\rfloor + 4(m-1)(n-1)$, with no hypothesis on the shape of the extremiser, so the leading constant and the order of the next term are theorems. The same count re-runs under the stricter separation.

3 Hyperedge networks: the vertex problem at $m=2$ and $m=3$, and a directed leading term

$\lfloor 2(n-1)/(r-1) \rfloor$ is a universal $m=3$ bound from a new incidence-graph rank lemma, attained for $r \geq 3$ when $2 \leq \binom{n-2}{r-2}$. One-way, the leading term is $(m-1)n^2/(4(r-1))$ under both separations.

What is left

- The undirected *vertex* problem at $m \geq 6$, open since 1974
- The decomposition that would turn the directed conjecture into a theorem: that every extremal non-hub digraph splits as $A \cup B$ with no arc coming back
- Whether the directed vertex value equals the arc value at $m \geq 3$, which Whitney's inequality does not hand over
- The exact values behind the leading terms now settled, one-way and for hyperedges alike

The map is filled in where the arguments reached,
and the blanks are marked where they did not.

AI: curse or blessing?

Pros

- $\text{\LaTeX}$ typesetting
- length & complexity of generated text

Cons

- length & complexity of generated text
 - control
-

It's a tool.

short leash + AI proofs = argued conjecture [AI]